Vintrabladet – Developer Guide

A walk-through of the Rainmaker test site for a developer who has never seen the code. Read it top to bottom once (about 45 minutes). After that, use section 9, "Where to look when something breaks" and section 10, "How to change things" as a reference.

The README explains *what* the site is and *why*. This guide explains *how the code works*, class by class.

Contents

1. The big picture
2. Running, testing and building
3. Folder map
4. What happens on a request
5. Java code, package by package
6. Templates (HTML)
7. Browser JavaScript and CSS
8. Content files and configuration
9. Where to look when something breaks
10. How to change things
11. Tests
12. Rules that must not be broken
13. Known weak spots
14. Glossary

1. The big picture

Vintrabladet is a fake Swedish local newspaper. Nothing on it is real. It exists so the Rainmaker team can build and test the Rainmaker widget (`rainmaker.js`) against a site that behaves like a real publisher's site, without using real, copyrighted articles.

The site does three jobs:

Job	Who uses it	Where in the code
Show news pages (front page, articles, sections)	People, and the Rainmaker widget running inside the pages	`web/PageController` + `templates/`
Offer machine-readable feeds (RSS, sitemap)	Rainmaker Content Ingestion (S5)	`web/FeedController` + `feed/`
Publish the "right answers" for the AI	The AI evaluation (S6)	`/eval/labels.csv` in `feed/FeedWriter`

It also imitates the third-party parts every Swedish news site has. These are the parts most likely to break a widget: cookie consent, late-loading ads, a tag manager, a strict security policy, a sticky menu and a paywall.

Five ideas that explain most of the code

1. **All content is loaded once, at start-up.** Articles, front page choices and ads are JSON files in `content/`. They are read and checked when the application starts. If one is wrong, the site *refuses to start* and names

the file and the field. This is called "fail fast". At runtime nothing is read from disk, and nothing can change.
2. **The server builds finished HTML.** Java collects data, and Thymeleaf templates turn it into HTML. There is no React and no client-side rendering. Rainmaker's Content Ingestion reads the HTML exactly as a crawler would.
3. **The browser adds only the "third-party" behaviour.** Cookie banner, ads, tag manager, sticky menu and paywall button are small JavaScript files. Each one is allowed to fail without breaking the page ("fail open").
4. **No visible text is hard-coded.** Every word of the interface is in `messages.properties`. The article text lives in the content files.
5. **Same input, same output.** Nothing depends on the current time or on randomness on the server. That keeps the tests stable. The only randomness is the delay before ads appear, and that happens in the browser.

Technology

Part	Choice	Why
Language	Java 21	The project standard (Getting Started Guide)
Framework	Spring Boot 3.5 (Spring MVC, embedded Tomcat)	Project standard
HTML	Thymeleaf templates	Server-side rendering; escapes text automatically
JSON	Jackson (comes with Spring Boot)	Reads the content files
XML	StAX (built into the JDK)	Writes RSS and sitemap
Build	Maven, through the Maven Wrapper (`mvnw`)	No Maven install needed
Style	google-java-format (Spotless) + Checkstyle (Google rules)	Project standard; enforced by `verify`
Tests	JUnit 5, AssertJ, MockMvc; `node --test` for the browser JS	Project standard

There are **no other dependencies**: only `spring-boot-starter-web`, `spring-boot-starter-thymeleaf` and `spring-boot-starter-test` (see `pom.xml`).

2. Running, testing and building

You need a **JDK 21 or newer**. Maven does not need to be installed, because `mvnw` downloads it the first time. Node.js 22.18+ is only needed for the JavaScript tests.

In PowerShell, write `.\mvnw.cmd` instead of `./mvnw`.

Command	What it does	Time
`./mvnw spring-boot:run`	Starts the site on http://localhost:3000	~5 s
`./mvnw test`	Runs all Java tests	~30 s
`./mvnw test -Dtest=ViewTest`	Runs one test class	
`./mvnw test -Dtest=PagesTest#sendsSecurityHeaders`	Runs one test method	
`./mvnw verify`	Tests **plus** the formatting check and Checkstyle. Run this before every push.	~45 s
`./mvnw spotless:apply`	Reformats all Java code to Google style	
`npm test`	Tests for the browser JavaScript	~1 s

Stop the site with **Ctrl+C**.

To use another port, set the environment variable first: `$env:PORT = '3100'` (PowerShell) or `PORT=3100 ./mvnw spring-boot:run` (bash).

When you change something

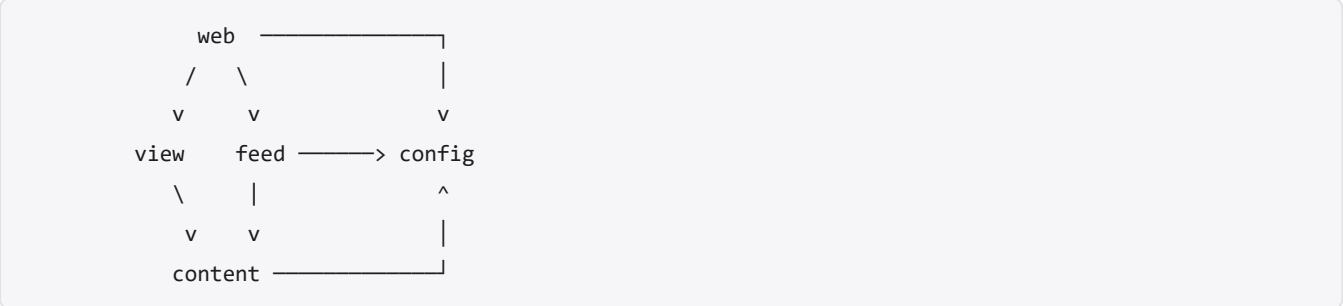
You changed...	Do this to see it
CSS, JS, or a template under `src/main/resources`	Restart the site. Static files are sent with `no-cache`, so a normal reload is enough after the restart.
Java code	Restart the site
A file in `content/`	Restart the site. Content is only read at start-up.

3. Folder map

```
rainmaker-test-site/
├── pom.xml                       Build: dependencies, Java version, Spotless, Checkstyle
├── mvnw, mvnw.cmd, .mvn/         Maven Wrapper; .mvn/jvm.config is needed by google-java-format
├── package.json                  Only "npm test" for the browser JS tests
├── content/                      THE CONTENT (not code)
│   ├── articles/vb-00NN-*.json   One file per article
│   ├── front-page.json           Lead story, "JUST NU" story, "Mest läst" list
│   └── ads.json                  Six fake advertisers
├── docs/                         This guide and the README screenshots
└── src/
    ├── main/java/nu/rainmaker/testsite/
    │   ├── TestSiteApplication.java   main(): starts Spring Boot
    │   ├── config/    Settings from application.yml / environment variables
    │   ├── content/   Reads and checks content/; the domain model (Article, ...)
    │   ├── view/      Prepares data for the templates (dates, ad slots, paywall split)
    │   ├── web/       URLs (controllers), filters (headers, logging), error pages
    │   └── feed/      RSS, sitemap, robots.txt, labels CSV, tag manager container
    ├── main/resources/
    │   ├── application.yml       Configuration
    │   ├── messages.properties   ALL visible interface text (Swedish)
    │   ├── templates/            Thymeleaf HTML templates
    │   └── static/               Served as /static/...: styles.css, favicon, js/
    └── test/
        ├── java/...              JUnit tests, same package layout as main
        └── javascript/           node --test tests for consent and tag manager
```

How the packages depend on each other

Arrows mean "uses". Nothing points back up, so `content` and `config` can be tested without a web server.



- `content` knows nothing about HTML or HTTP.
- `view` knows about pages, but not about URLs.
- `web` is the only package that knows about HTTP.

4. What happens on a request

4.1 At start-up (once)

```
main()  (TestSiteApplication)
  │
  ├─ Spring reads application.yml + environment variables
  │    └─> SiteProperties (config)          checks base-url, widget URL, install method
  │
  ├─ ContentConfiguration (content) creates three beans:
  │    ├─ articleRepository()   reads content/articles/*.json
  │    │     └─ ArticleParser.parse(...) for each file   ← stops start-up on any error
  │    ├─ frontPage(...)        reads content/front-page.json, checks the ids exist
  │    └─ adInventory()         reads content/ads.json
  │
  ├─ SecurityHeadersFilter builds the CSP header text once
  ├─ FeedController builds the TagContainer once
  └─ Tomcat starts listening on PORT (3000)
```

When you see `Loaded 12 articles from ...` and `Tomcat started on port 3000` in the log, everything above has worked.

4.2 For every request, e.g. `GET /artikel/vintraloppet-firar-25-ar`

```
Browser
  |
  v
RequestLoggingFilter      puts trace_id + correlation_id in the log context (MDC)
  |                       starts a timer
  v
SecurityHeadersFilter     adds CSP, X-Content-Type-Options, Referrer-Policy, X-Robots-Tag
  |
  v
Spring MVC picks a handler by URL:
  /                 → PageController.frontPage
  /artikel/{slug}   → PageController.article       ← this one
  /sektion/{id}     → PageController.section
  /rss.xml etc.     → FeedController
  /static/**        → Spring's static file handler (src/main/resources/static)
  anything else     → NoHandlerFoundException → 404 page
  |
  v
LayoutModelAdvice         adds "sections", "site", "rainmaker" to the model (all HTML pages)
  |
  v
PageController.article
  ├─ articles.findBySlug(slug)        not found → PageNotFoundException → 404 page
  ├─ articleBodyBuilder.build(article) inserts the in-article ad, splits the paywall
  ├─ articles.related(article, 4)      "Läs också"
  ├─ adSlots.slot(...)                 panorama + skyscraper ads
  └─ returns "article"  → templates/article.html
  |
  v
Thymeleaf renders article.html inside layout.html, using fragments.html
  |
  v
RequestLoggingFilter (finally) writes ONE log line: method, path, status, duration_ms
  |
  v
Browser receives finished HTML, then loads:
  tcf-stub.js (sync) → site.js (module: consent, sticky header, paywall, ads)
  → tag-manager.js (async) → rainmaker.js (async, only if configured)
```

If any step throws, `ErrorPageAdvice` turns the exception into the right error page (see 4.3).

4.3 What the site answers when something goes wrong

Situation	Exception	Answer	Logged?
Unknown article or section	`PageNotFoundException`	404, HTML page "Sidan finns inte" with the normal layout	Only the request line

Situation	Exception	Answer	Logged?
Unknown URL	`NoHandlerFoundException`	Same 404 page	Only the request line
Missing file under `/static/`	`NoResourceFoundException`	404, **plain text** (a browser loading CSS has no use for HTML)	Only the request line
Wrong HTTP method (e.g. POST)	a `ServletException` that is also an `ErrorResponse`	405 + `Allow: GET, HEAD` header, plain text	Only the request line
Any other bug	any `Exception`	500, HTML page "Något gick fel". **No stack trace is shown to the reader.**	**Yes,** `ERROR` **with the stack trace**
Rare: a `ServletException` without a status	`ServletException`	500, plain text "Sidan kunde inte visas just nu..."	**Yes,** `ERROR`

5. Java code, package by package

All classes are in `src/main/java/nu/rainmaker/testsite/`. "Bean" means an object that Spring creates once and hands to the classes that ask for it in their constructor. That is constructor injection, which the project rules require.

5.1 `TestSiteApplication`

TestSiteApplication.java holds the `main` method.

- `@SpringBootApplication` turns on component scanning: every `@Component`, `@Controller` and `@Configuration` under this package becomes a bean.
- `@ConfigurationPropertiesScan` finds `SiteProperties`.

You will rarely touch this class.

5.2 Package `config`: settings

`SiteProperties`

SiteProperties.java

A Java `record` filled from the `site:` block in `application.yml`. Spring converts `base-url` to `baseUrl` and so on.

Part	What it does
`baseUrl`	Public address. Trailing slashes are removed. It must be `http://` or `https://` with a host, otherwise start-up fails.
`contentDirectory`	Where `articles/`, `front-page.json` and `ads.json` are. A relative path is relative to the folder you start from.
`absoluteUrl(path)`	`baseUrl + path`. Used for links in RSS and the sitemap.
`Rainmaker` (nested record)	`scriptUrl` + `installMethod`. A blank `scriptUrl` means "no widget". The URL must be **https**, or plain http on `localhost` / `127.0.0.1` only.
`Rainmaker.origin()`	E.g. `https://cdn.rainmaker.nu`. Added to the CSP so the browser may load the widget.
`loadedByTemplate()` / `loadedByTagManager()`	Exactly one is true when a widget is configured, and both are false when none is.
`InstallMethod`	`TEMPLATE` (`template` in yml) or `TAG_MANAGER` (`tag-manager`).

Important: all checks happen in the record's *compact constructor* (`public SiteProperties { ... }`). A wrong value therefore stops the application at start-up with a clear message, instead of failing later.

5.3 Package `content`: the content and its rules

This package is "the domain". It has no Spring web code and no HTML.

Model classes (records and enums)

Class	What it is
`Article`	One article. It has `id`, `slug`, `section`, `expectedZone`, `premium`, `kicker`, `headline`, `lead`, `author`, `publishedAt`, `updatedAt` (may be null) and `body`. Helpers: `path()` returns `/artikel/<slug>`; `lastUpdate()` returns an Optional; `lastModified()` returns the update time, or the publish time if there is none.
`Article.Author`	`name`, `role`
`ArticleBlock`	A **sealed interface** with exactly four kinds of body block: `Paragraph(text)`, `Subheading(text)`, `Quote(text, source)` and `Factbox(title, items)`. `type()` gives the name used in JSON and templates (`"paragraph"`, ...). `texts()` gives all words, for the reading time.
`Section`	Enum of the 9 sections **in menu order**. `id()` is used in URLs and JSON (`resor`). `messageKey()` gives the translation key (`section.resor`). The visible name is never in Java.
`BrandSafetyZone`	`GREEN`, `YELLOW`, `RED`: Rainmaker's zones (architecture UC-03)
`AdCreative`	One fake ad. It has `id`, `advertiser`, `headline`, `text`, `callToAction`, `theme` and `sections`. `THEMES` lists the colour themes that exist in CSS. `targets(section)` checks whether the ad is aimed at a section.
`FrontPage`	The editor's choices: `lead`, `breaking` (null means no "JUST NU" bar) and `mostRead`

All lists inside records are copied with `List.copyOf`, so they can never be changed after creation. Everything here is **immutable**, which is why it is safe to share between requests without locking.

Parsing and validation

Class	What it does
`ContentConfiguration`	`@Configuration` with three `@Bean` methods: `articleRepository()`, `frontPage(...)` and `adInventory()`. It reads the files with Jackson's `ObjectMapper.readTree`. Article files are read in **file-name order**. It logs `Loaded N articles from ...`.
`ArticleParser`	Turns one JSON tree into an `Article` and checks every field. `SLUG_PATTERN` allows lower-case `a-z`, digits and single hyphens. Dates must have a time zone offset (`Z` or `+02:00`), so they never depend on the server's zone. The body must contain at least one paragraph. `premium` defaults to `false`.
`JsonFields`	Small helpers such as `requireText` and `requireObject`, which throw with "file: field" messages. Package-private.
`ContentValidationException`	The message is always `<file>: <problem>`, e.g. `vb-0005-kantareller.json: "section" is not a known section: matt`

`FrontPage.parse` and `AdInventory.parse` follow the same pattern. `FrontPage` also checks that every article id exists and that "Mest läst" has no duplicates.

`ArticleRepository`

ArticleRepository.java

Holds all articles **sorted newest first** by `publishedAt`. The sort uses the actual moment in time, so `+02:00` and `Z` compare correctly. It fails at creation if two articles share an id or a slug.

Method	Used for
`all()`	Front page lists, RSS, sitemap, labels

Method	Used for
`findById(id)`	Front page config
`findBySlug(slug)`	Article page
`inSection(section)`	Section page
`related(article, limit)`	"Läs också": same section first, then the newest from other sections. Never the article itself.
`latestPublishedAt()`	RSS `lastBuildDate`

With 12 articles, a simple list search is fast enough. A map would only be needed with thousands of articles.

`AdInventory.pick(section, slotNumber)`, the ad rule

AdInventory.java

```
ordered = [ads that target this section, in file order] + [all other ads, in file order]
ad      = ordered[ slotNumber mod ordered.size ]
```

- On an article in `resor`, slot 0 gets the first travel ad, and slot 1 gets the second travel ad (or the first non-travel ad if there is only one).
- On the front page `section` is `null`, so it is simply file order.
- It is **deterministic**: the same page always gets the same ads, which keeps the tests stable.

5.4 Package `view`: preparing data for the templates

`ViewHelper` (bean name `view`)

ViewHelper.java

Templates call it as `${@view.time(...)}`. The `@` means "a Spring bean".

Method	Example	Note
`date(t)`	`13 okt. 2026`	Always in **Europe/Stockholm** time, even if the server runs in UTC
`dayAndMonth(t)`	`13 okt.`	
`time(t)`	`09.42`	Swedish newspapers use a dot, not a colon
`iso(t)`	`2026-10-13T09:42:00+02:00`	For `<time datetime="...">`
`readingTimeMinutes(article)`	`3`	Words in lead + body, 200 words/minute, rounded **up**, at least 1
`initials(name)`	`AH`	Round avatar in the byline
`showKicker(article, sectionName)`	`false`	Hides the kicker if it just repeats the section ("RESOR RESOR")

`AdSlot`, `AdSlots`

AdSlot.java · AdSlots.java

`AdSlot(format, creativeId, reserveSpace)` describes an **empty** ad box.

- Formats: `PANORAMA` (wide, top), `RECTANGLE` (in the text) and `SKYSCRAPER` (right column).
- The ad's *text* is never in the HTML. Only `creativeId` is, and the browser fetches the text later (see `ads.js`).
- `AdSlots.slot(...)` asks `AdInventory.pick` which ad to use.

Slot numbers used by `PageController`:

Page	Slot 0	Slot 1	Slot 2
Front page (section = null)	panorama	skyscraper	–
Article (section = the article's)	panorama	in-article rectangle (**no reserved space**)	skyscraper
Section page	panorama	–	–

`ArticleBodyBuilder` and `ArticleBody`

ArticleBodyBuilder.java · ArticleBody.java

This is the most "clever" code on the site. Read it carefully before changing the ad or paywall placement.

```
Input: article.body = [B0, B1, B2, B3, B4, B5, B6]   (7 blocks), premium = true

Step 1 – in-article ad (only if the article has at least 5 blocks):
         [B0, B1, B2, AD, B3, B4, B5, B6]          AD inserted at index 3

Step 2 – paywall (only if premium AND more than 4 blocks):
         split = 4 + 1 (because the ad is before the split point) = 5
         free   = [B0, B1, B2, AD, B3]
         locked = [B4, B5, B6]
```

Constants: `AD_AFTER_BLOCKS = 3` and `FREE_BLOCKS = 4`. The result is an `ArticleBody(free, locked)` of `Item`s. Each item is either a block or an ad (`isAd()`). `paywalled()` is true when `locked` is non-empty.

Why the in-article ad reserves no space: many real sites get this wrong, and the text jumps down when the ad arrives. We keep one such slot on purpose, so Rainmaker can be tested for layout shift (CLS).

5.5 Package `web`: URLs, filters and errors

`PageController`, the HTML pages

PageController.java

URL	Method	Template	Model attributes it sets
`/`	`frontPage`	`front.html`	`frontPage`, `secondaryStories` (2), `moreStories`, `latest` (6), `panoramaAd`, `skyscraperAd`
`/artikel/{slug}`	`article`	`article.html`	`article`, `sectionName`, `showKicker`, `body`, `related` (4), `panoramaAd`, `skyscraperAd`
`/sektion/{sectionId}`	`section`	`section.html`	`section`, `sectionName`, `sectionArticles`, `panoramaAd`

Every page also sets `pageTitle`, `description`, `path` (for the canonical link) and `activeNav` (which menu item is highlighted). The controller only **collects data**, and all HTML is in the templates.

The front page: the lead story comes from `front-page.json`. The next two newest articles become the "secondary" stories, and the rest go to "Fler nyheter".

`LayoutAttributes` and `LayoutModelAdvice`

LayoutAttributes.java · LayoutModelAdvice.java

Every page needs the menu (`sections`), the settings (`site`) and the widget config (`rainmaker`).

- `LayoutModelAdvice` adds them before **every** `@Controller` method.
- Error handlers do not get `@ModelAttribute`s from Spring, so `ErrorPageAdvice` adds the same map itself through `LayoutAttributes`.

That is why the map lives in its own class.

`ErrorPageAdvice`

ErrorPageAdvice.java

`@ControllerAdvice` with four `@ExceptionHandler`s (see the table in 4.3). Spring picks the **most specific** matching handler, so `NoResourceFoundException` (a subclass of `ServletException`) goes to `staticFileNotFound`, not to `requestError`.

- `unexpected(Exception)` is the safety net. It logs at `ERROR` with the stack trace and shows `status-page.html`.
- **If a page shows "Något gick fel", the real reason is in the log, never on the page.**

`PageNotFoundException`

PageNotFoundException.java

Throw it from a controller to show the 404 page.

`FeedController`, machine-readable URLs

FeedController.java

URL	Returns	Used by
`/rss.xml`	RSS 2.0, all articles	Content Ingestion (S5)
`/sitemap.xml`	Google News sitemap, all articles	S5
`/robots.txt`	Allows everything, points to the sitemap	Crawlers
`/eval/labels.csv`	`article_id,expected_zone,section,url`	AI evaluation (S6)
`/ads/creatives.json`	All ads (Jackson serialises the records)	`static/js/ads.js`
`/tag-manager/container.json`	`TagContainer`	`static/js/tag-manager.js`
`/health`	ok	Uptime checks

`SecurityHeadersFilter`

SecurityHeadersFilter.java

Adds these headers to **every** response:

- `Content-Security-Policy`
- `X-Content-Type-Options: nosniff`

- `Referrer-Policy`
- `X-Robots-Tag: noindex`

The CSP only allows the site's own files, plus the widget's origin when one is configured. Without a widget it looks like this:

```
default-src 'self'; script-src 'self'; connect-src 'self'; img-src 'self' data:;
style-src 'self'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'
```

Consequence: inline `<script>` and `style="..."` are blocked by the browser. All JS and CSS must be in files under `static/`.

`RequestLoggingFilter`

RequestLoggingFilter.java

Runs first (`@Order(HIGHEST_PRECEDENCE)`).

- It sets `trace_id` and `correlation_id` in the SLF4J **MDC**, so every log line written during the request carries them.
- A valid W3C `traceparent` header is continued. A missing or invalid one gets a new random trace id.
- In `finally` (so also on errors), it writes one log line per request.
- It logs the **path only**: never query strings, IPs or other reader data.

A real log line, from `GET /artikel/vintraloppet-firar-25-ar`:

```
{"@timestamp":"2026-10-14T07:40:12.203636700Z","log":
{"level":"INFO","logger":"nu.rainmaker.testsite.web.RequestLoggingFilter"},
 "message":"Request
handled","correlation_id":"1fac1ed3-...","trace_id":"4bf92f3577b34da6a3ce929d0e0e4736",
 "event":{"name":"http.request"},"http":{"request":{"method":"GET"},"response":{"status_code":200}},
 "url":{"path":"/artikel/vintraloppet-firar-25-ar"},"duration_ms":307, ...}
```

The JSON (ECS) format comes from `logging.structured.format.console: ecs` in `application.yml`. No logging library was added for it.

5.6 Package `feed`: RSS, sitemap and friends

Class	What it does
`FeedWriter`	`rss()`, `newsSitemap()`, `robotsTxt()` and `evaluationLabels()`. Uses **no current time**: `lastBuildDate` is the newest article's date. `csvField` quotes CSV values that contain `,` `"` or line breaks. `rfc822` writes RSS dates in GMT.
`XmlDocument`	A small wrapper around the JDK's StAX `XMLStreamWriter`, with indentation. StAX escapes `&`, `<` and so on itself, so the XML is always valid, whatever is in an article.
`TagContainer`	The fake tag manager's tag list. It always contains `vintra-stats` (trigger `consent`). It adds `rainmaker` (trigger `pageview`) only when `RAINMAKER_INSTALL_METHOD=tag-manager`.

6. Templates (HTML)

Folder: `src/main/resources/templates/`

File	What it is
`layout.html`	The frame of every page: `<head>`, fiction banner, logo, sticky menu, footer, cookie banner and the **Rainmaker script slot**. A page plugs into it with `th:replace="~{layout :: page(~{::content})}"`.
`fragments.html`	Reusable pieces: `card`, `compactCard`, `shortTime`, `publishTimes`, `plusBadge`, `illustration` (SVG per section), `adSlot`, `articleBlock`, `bodyItem` and `consentBanner`
`front.html`	Front page
`article.html`	Article page, including the paywall
`section.html`	Section page
`status-page.html`	404 and 500 pages

Thymeleaf in 60 seconds

Syntax	Meaning
`th:text="${article.headline()}"`	Insert a value, **HTML-escaped** (safe against XSS)
`th:text="#{front.latest}"`	Insert text from `messages.properties`
`#{article.published(${a}, ${b})}`	Text with `{0}`, `{1}` placeholders
`${@view.time(x)}`	Call a method on the Spring bean named `view` (`ViewHelper`)
`th:if`, `th:unless`, `th:each`, `th:switch`/`th:case`	Condition, negated condition, loop, switch
`th:replace="~{fragments :: card(${story}, 'standard')}"`	Insert a fragment
`<th:block>`	An invisible wrapper element
`<!--/* ... */-->`	A comment only for developers, removed before sending

Gotcha: on the same element, `th:replace` runs *before* `th:if` and `th:each`, so the condition would be ignored. That is why the templates wrap conditional fragments:

```
<th:block th:if="${article.premium()}"><th:block th:replace="~{fragments :: plusBadge}"></th:block>
</th:block>
```

HTML rules Rainmaker depends on (tested in `PagesTest`)

- Only the article itself is inside `<article class="article">`. Menus, "Läs också" and the sidebar are outside.
- The kicker, byline, ad slots and paywall offer contain **no** `<p>`. Content Ingestion reads `article p` as body text, so a `<p>` there would leak into it.
- `<h1 data-rainmaker-headline>` marks the headline the widget hooks on (UC-06).
- `<article data-article-id="vb-0001">` gives the article id.
- The `expectedZone` is **never** in the HTML. Otherwise the AI classifier could "cheat".
- PLUS articles contain the **full** text. CSS hides the locked part, and `data-paywall-state="locked"` turns it on.

7. Browser JavaScript and CSS

Folder: `src/main/resources/static/js/`, served as `/static/js/...`.

7.1 Load order (from `layout.html`)

```
<head>
  tcf-stub.js        SYNC   – must define window.__tcfapi before anything else runs
  site.js            module – runs after the HTML is parsed; imports the four features
  tag-manager.js     async  – runs whenever it has downloaded
</head>
<body>
  ...
  rainmaker.js       async  – only when RAINMAKER_SCRIPT_URL is set and install method = template
</body>
```

7.2 Each file

File	Type	What it does	If it fails
`tcf-stub.js`	plain script	Defines a placeholder `window.__tcfapi` that **queues** calls, answers `ping` right away, and adds the hidden `__tcfapiLocator` iframe (TCF spec)	The consent API does not exist, and the tag manager never fires consent tags
`site.js`	module	Starts consent, sticky header, paywall and ads, each in its own `try` / `catch`, so one failure does not stop the others	Logs `[vintrabladet] <feature> failed` in the console
`consent-core.js`	module, **no DOM**	The full fake `__tcfapi`: `ping`, `addEventListener`, `removeEventListener`, `getTCData`. `buildTcData` makes the TCData object. `takeOverFromStub` replays the queue. Saves the choice in `localStorage` under `vintrabladet.consent`.	Unit-tested in Node
`consent.js`	module	Connects the core to the page: shows the banner if no choice is saved, handles the two buttons, pushes `consent_update` to `dataLayer`, and reopens the banner from the "Cookieinställningar" link	Banner never shows
`ads.js`	module	Fetches `/ads/creatives.json`, then fills each `[data-ad-slot]` after a random **0.3–1.5 s**. Builds the ad with `textContent` only (no `innerHTML`). Sets `data-ad-state` to `loading`, `filled`, `empty` or `failed`.	Slots stay empty, and the page works
`paywall.js`	module	The "Fortsätt läsa (testläge)" button sets `data-paywall-state="unlocked"`	Text stays faded
`sticky-header.js`	module	`IntersectionObserver` adds `is-stuck` to the menu when the big logo scrolls away, which shows the mini logo	No mini logo; the menu still sticks (that part is CSS)
`tag-manager.js`	plain script	A mini GTM. Fetches `container.json` and injects each tag as an async `<script>`. `pageview` tags load right away. `consent` tags load only once `purpose.consents[1] === true`. Writes events to `window.dataLayer`.	No tags load

File	Type	What it does	If it fails
`vintra-stats.js`	plain script	Fake analytics. Pushes `page_view` and sets `data-stats-loaded` on `<html>`. Sends nothing anywhere.	–
`widget-stub.js`	plain script	A **stand-in for rainmaker.js** to test the *installation*. Sets `data-rainmaker-loaded` and `data-rainmaker-consent`, and logs what it found.	–

7.3 Useful things to check in the browser (F12)

What	Where
Consent state	Console: `__tcfapi('getTCData', 2, console.log)`
Forget the cookie choice	Console: `localStorage.removeItem('vintrabladet.consent')`, then reload
Tag manager events	Console: `dataLayer`
Was the stats tag loaded?	`<html data-stats-loaded="true">` (only after "Godkänn alla")
Was the widget loaded?	`<html data-rainmaker-loaded="true">`
Ad state	`data-ad-state` on each `<aside class="ad-slot">`
CSP blocked something	The console shows "Refused to load ... Content Security Policy"

7.4 CSS

`styles.css` is one file, with numbered sections you can search for:

Section	What it covers
`/* 1. Design tokens`	Colours, fonts and spacing as CSS variables. **Change colours here.**
`/* 2. Base`	Base element styles and `.button`
`/* 3.`	Fiction banner, header, sticky navigation, footer
`/* 4.`	Cards and section illustrations (one colour per section)
`/* 5a–5d`	Front page, section page, article page, status pages
`/* 6. Publisher realism`	"JUST NU", "Mest läst", ad slots, paywall fade, consent banner
`/* 7. Responsive`	Mobile layout

Class names follow **BEM**: `block__element--modifier`, e.g. `card__headline` and `card--lead`.

8. Content files and configuration

8.1 An article file

`content/articles/vb-0006-badrumsrenovering.json` (shortened):

```
{
  "id": "vb-0006",                                  // unique; a-z, 0-9, hyphens
  "slug": "badrumsrenovering-experternas-rad",      // unique; becomes /artikel/<slug>
  "section": "bostad",                              // one of the Section ids
  "expectedZone": "green",                          // green | yellow | red – NOT shown on the page
  "premium": true,                                  // optional, default false – PLUS paywall
  "kicker": "RENOVERING",
  "headline": "Badrummet som håller i 40 år – ...",
  "lead": "Ett nytt badrum kostar ofta ...",
  "author": { "name": "Frida Holm", "role": "Bostadsreporter" },
  "publishedAt": "2026-10-10T07:45:00+02:00",       // MUST have an offset (+02:00 or Z)
  "updatedAt": null,                                // optional
  "body": [
    { "type": "paragraph",  "text": "..." },
    { "type": "subheading", "text": "..." },
    { "type": "quote",      "text": "...", "source": "..." },
    { "type": "factbox",    "title": "...", "items": ["...", "..."] }
  ]
}
```

(Real JSON does not allow `//` comments. They are only here to explain.)

Text rules for content: everything must be fictional. Invent people and companies, and use the town of Vintra. Real places are only used for travel destinations.

8.2 `front-page.json` and `ads.json`

```
{ "leadArticleId": "vb-0001", "breakingArticleId": "vb-0003",
  "mostReadArticleIds": ["vb-0002", "vb-0003", "vb-0005", "vb-0001", "vb-0008"] }
```

`breakingArticleId` may be left out, and then there is no "JUST NU" bar.

`ads.json` holds `{"creatives": [ ... ]}`. Each creative has:

- `id`, `advertiser`, `headline`, `text`, `callToAction`
- `theme`: one of `sunset`, `forest`, `night`, `clay`, `ocean`, `berry`
- `sections`: a list of section ids

8.3 Configuration and environment variables

All settings are in `application.yml`. The syntax `${NAME:default}` reads an environment variable, or uses the default.

Environment variable	Default	Effect
`PORT`	`3000`	Server port
`BASE_URL`	`http://localhost:<PORT>`	Absolute links in RSS/sitemap/labels
`CONTENT_DIRECTORY`	`content`	Where the content files are
`RAINMAKER_SCRIPT_URL`	*(empty = no widget)*	Address of rainmaker.js
`RAINMAKER_INSTALL_METHOD`	`template`	`template` or `tag-manager`

Fixed settings worth knowing:

- `spring.web.locale: sv` with `locale-resolver: fixed` makes the site always Swedish, whatever language the browser asks for.
- `spring.mvc.static-path-pattern: /static/**` decides the URL of static files.

Try the widget installation with the stub (PowerShell):

```
$env:RAINMAKER_SCRIPT_URL = 'http://localhost:3000/static/js/widget-stub.js'
$env:RAINMAKER_INSTALL_METHOD = 'tag-manager'   # leave out for the template install
.\mvnw.cmd spring-boot:run
```

Remove them again with `Remove-Item Env:RAINMAKER_SCRIPT_URL, Env:RAINMAKER_INSTALL_METHOD`.

There are no secrets in this project, and there must never be. Nothing here needs a password or API key.

9. Where to look when something breaks

9.1 The site does not start

Read the log from the **bottom up**. Look for `ContentValidationException` or `IllegalArgumentException`. The text after it names the file or setting.

Message (example)	Cause	Fix
`vb-0005-kantareller.json: "section" is not a known section: matt`	Typo in a content file	Fix the file. The valid values are in `Section.java`.
`...json: invalid JSON (Unexpected character ('' (code 65279 / 0xfeff))`	The file was saved **with a BOM**. Windows PowerShell 5.1 `Set-Content -Encoding utf8` and some editors do this.	Save as "UTF-8" (not "UTF-8 with BOM"). In VS Code, click the encoding in the status bar.
`...json: invalid JSON (Unexpected character ...)`	A missing comma or quote, etc.	Check the JSON at the line Jackson mentions
`"publishedAt" must be an ISO date-time with offset`	A date without `+02:00` / `Z`	Add the offset
`front-page.json: "leadArticleId" refers to unknown article "vb-0099"`	The front page points to an id that does not exist	Fix the id
`articles: duplicate slug: ...`	Two articles share a slug or id	Make them unique
`site.rainmaker.script-url must use https (http only for localhost)`	Wrong widget URL	Use https
`Port 3000 was already in use`	The site, or something else, is already running	Stop the other process, or set `PORT`
`release version 21 not supported` / `invalid target release`	Your JDK is older than 21	Install JDK 21+ and check `java -version`

9.2 Symptom → where to look

Symptom	Look first in
A page shows **"Något gick fel"** (500)	The **log**: search for `"level":"ERROR"`. The stack trace names the class and line. Often a template calls a method that does not exist, or a null value.
A page shows **"Sidan finns inte"** (404) but should exist	`PageController` (URL mapping), the slug in the JSON file, and `Section` for `/sektion/...`
Wrong or missing text, or `??some.key_sv??` on the page	The key is missing in `messages.properties`, or there is a typo in the template's `#{...}`
Wrong date or time on a page	`ViewHelper` (time zone, formats) and the `publishedAt` offset in the content file
Wrong reading time	`ViewHelper.readingTimeMinutes`
Ad in the wrong place, or paywall cut at the wrong point	`ArticleBodyBuilder` (constants at the top)
Wrong ad shown	`AdInventory.pick` + `sections` in `ads.json` + the slot numbers in `PageController`
Ads never appear	Browser console + Network tab: does `/ads/creatives.json` load? Check `data-ad-state` (`failed` / `empty`) and `ads.js`

Symptom	Look first in
Cookie banner does not show / always shows	`localStorage['vintrabladet.consent']`, then `consent.js` and `consent-core.js`
Stats tag loads before consent	`tag-manager.js` (`whenConsentGiven`) and `TagContainer` triggers
Widget not loaded	`RAINMAKER_SCRIPT_URL` is set? The CSP header includes its origin? The `th:if` at the bottom of `layout.html`, `TagContainer.forSite`
Browser console: "Refused to load ... Content Security Policy"	`SecurityHeadersFilter`. Either inline JS/CSS was added (not allowed), or a new external origin needs adding.
A CSS/JS file gives 404	The path must start with `/static/` and the file must be in `src/main/resources/static/`
Layout looks broken on mobile	`styles.css` section `/* 7. Responsive`
RSS/sitemap wrong	`FeedWriter`, plus `BASE_URL` for the absolute links
`./mvnw verify` fails with `spotless`	Run `./mvnw spotless:apply`, then verify again
`./mvnw verify` fails with Checkstyle	The message gives the file:line and the rule, e.g. a Javadoc summary must end with a period
A test fails	Read the **AssertJ message**: it shows expected vs. actual. Then run only that test with `-Dtest=Class#method`.

9.3 Following one request through the log

Every request writes one `"message":"Request handled"` line with `url.path`, `http.response.status_code` and `duration_ms`. If the request failed, an `ERROR` line with **the same** `trace_id` comes just before it. Search the log for that `trace_id` to see everything about that request.

If a caller sends a `traceparent` header, its trace id is reused. A request can then be followed across services, which is how the real Rainmaker services will do it.

10. How to change things

Always: create a branch, write or adjust the test first, change the code, run `./mvnw verify`, then open a PR.

Add an article

1. Copy an existing file in `content/articles/`, and name it `vb-00NN-short-name.json`.
2. Change `id`, `slug` (both unique), `section`, `expectedZone`, the texts and the dates.
3. Start the site. If something is wrong, the log says what. Then run `./mvnw test`.
4. Some tests check the content set as a whole:
    - `ContentFilesTest` expects **8–12** articles, all three zones, and some (but not all) PLUS articles.
    - `PagesTest` has a list of slugs that must cover every article. Update them if you go beyond these limits.

Change the front page

Edit `content/front-page.json`. No code changes are needed.

Add an ad

Add an object to `content/ads.json`. Use an existing `theme`, or add a new theme to **both** `AdCreative.THEMES` and `styles.css` (`.ad-creative--<theme>`).

Add a section

1. Add the enum constant to `Section.java`. Its position is its place in the menu.
2. Add `section.<id>=Visible Name` to `messages.properties`.
3. Add an illustration `<g th:case="'<id>'">` in `fragments.html`, plus colours `.illustration--<id>` / `.card--<id>` in `styles.css`.
4. `ContentFilesTest.everySectionHasTranslatedName` fails if you forget step 2.

Add a new block type (e.g. an image)

1. Add a record to the sealed interface `ArticleBlock`. The compiler then shows you every `switch` that must handle it.
2. Add a `case` in `ArticleParser.parseBlock`.
3. Add a `th:case` in `fragments.html` → `articleBlock`.
4. Add tests in `ArticleParserTest` and `PagesTest`.

Add a new page/URL

1. Add a method to `PageController` (or a new `@Controller`). Set `pageTitle`, `description`, `path` and `activeNav`.
2. Create `templates/<name>.html` using the layout (copy `section.html`).
3. Put all visible text in `messages.properties`.
4. Add a MockMvc test. `PagesTest.allPages()` makes it go through the fiction-banner and noindex checks automatically.

Add a machine-readable output

Put the logic in `feed/`, and add a `@GetMapping` in `FeedController` with the right content type. Use `XmlDocument` for XML; never build XML with string concatenation.

Change interface text, or add English

- All text is in `messages.properties`.
- In texts **with** `{0}` placeholders, a single quote must be written twice (`''`).
- For English, create `messages_en.properties` with the same keys, and change `spring.web.locale` (it is fixed to `sv` today).

Change where the ad or paywall goes

Change `AD_AFTER_BLOCKS` / `FREE_BLOCKS` in `ArticleBodyBuilder`, and update `ViewTest`, which tests the exact positions.

Add browser behaviour

1. Write a new module in `static/js/`, exporting an `init...()` function.
2. Add it to the `features` list in `site.js`.
3. Use **no** `innerHTML` (`StaticFilesTest` checks this), and no inline scripts (the CSP blocks them).
4. Add the file to the list in `StaticFilesTest`.

Load a new external script or service

Add its origin to `SecurityHeadersFilter`, otherwise the browser blocks it. Think twice: the strict CSP is part of what we are testing.

11. Tests

About 180 Java tests and 27 JavaScript tests. They are fast and deterministic: no sleeps, no network and no wall clock.

Test class	Covers
`config/SitePropertiesTest`	URL checks, defaults, widget origin, install method
`content/ArticleParserTest`	Valid articles + one parameterised case per broken field (names file + field)
`content/ArticleRepositoryTest`	Sorting (also across time zones), look-ups, "related", duplicates
`content/FrontPageAndAdsTest`	front-page.json and ads.json parsing; the ad picking rule
`content/ContentFilesTest`	The **real** files in `content/` : count, zones, PLUS mix, dates, translations
`view/ViewTest`	Swedish time (incl. the switch to winter time), reading time, initials, kicker, ad + paywall positions
`web/PagesTest`	Every page via MockMvc: fiction label, noindex, security headers, front page choices, Rainmaker markup, zone never leaked, paywall, ads, consent banner, script order, sections
`web/ErrorHandlingTest`	404 page, plain 404 for static files (incl. path traversal attempts), 405 + Allow, 500 page without a stack trace
`web/FeedsTest`	RSS and sitemap parsed as real XML, robots, labels CSV, ads JSON, tag container, health
`web/RainmakerInstallTest`	Template install vs tag-manager install (nested classes with their own properties)
`web/RequestLoggingFilterTest`	`traceparent` parsing
`web/StaticFilesTest`	Every JS file is served as JavaScript, imports exist, no raw HTML writes
`src/test/javascript/consent.test.ts`	The `__tcfapi` behaviour, stored choice, stub hand-over
`src/test/javascript/tag-manager.test.ts`	Consent-gated loading in the tag manager

Helpers:

- `ContentTestData` builds test articles.
- `WebTestSupport` has `fetch`, `page`, `body` (whitespace collapsed) and `escaped` (HTML-escaped text to search for).

`@SpringBootTest` tests start the whole application once. Tests with the same properties share that context, so they are not slow.

12. Rules that must not be broken

These come from the architecture and the project's `CLAUDE.md`, and most of them are checked by tests:

1. **Everything is fictional and labelled.** Every page has the fiction banner and `noindex`. No real people, companies or news.
2. `expectedZone` **never appears in the HTML.** Only in `/eval/labels.csv`.
3. **Fail open.** Any browser feature (and the Rainmaker widget) may fail without breaking the article.
4. **Respect consent.** Consent-gated tags load only after "Godkänn alla".
5. **No personal data in logs.** Log the path only: never query strings, IPs or reader input.
6. **No hard-coded UI text.** Use `messages.properties`.
7. **No inline JS/CSS, and no** `innerHTML`**.** The CSP and XSS safety depend on it.
8. **Deterministic output.** No `now()` and no randomness on the server.
9. **No secrets in the repo, ever.**
10. **Google Java Style and constructor injection.** `./mvnw verify` must pass.

13. Known weak spots

These are honest notes for the next developer. None of them affects the site today.

- **Content files with a UTF-8 BOM are rejected** (see 9.1). That is safe, but the message is cryptic. A possible improvement is to strip a leading BOM in `ContentConfiguration.readJson`.
- **A few comments in the JavaScript still refer to the earlier Node version of the site:**
  - `consent-core.js` mentions `test/consent.test.ts` (now `src/test/javascript/`).
  - `consent.js` mentions `render/consent.ts` (now `templates/fragments.html`).
  - `widget-stub.js` says `npm start` (now `.\mvnw.cmd spring-boot:run`). These should be fixed in a small separate PR.
- **The fake CMP is not a real CMP.** The `tcString` is readable text (`FAKE-TCF.all`) and there is no vendor list. A widget that decodes a real TC string would need a real CMP to test against.
- **Content is read only at start-up.** Editing an article needs a restart. This is intentional, because it keeps the site simple and deterministic.
- **Only Swedish.** The structure for English exists (`messages_en.properties`), but the locale is fixed to `sv`.
- **No metrics or dashboards.** The project's Definition of Done requires them for *services*. This is a test fixture, not a service, so only structured logs are in place.

14. Glossary

Term	Meaning
Kicker ("vinjett")	The small label above a headline, e.g. "RESOR"
Lead ("ingress")	The bold introduction under the headline
Byline	Author name and role
Factbox ("faktaruta")	Box with a title and bullet points
PLUS	Paid articles behind a paywall
JUST NU	Red breaking-news bar on the front page
Panorama / rectangle ("modul") / skyscraper	Common Swedish ad formats: wide top banner / box in the text / tall box in the side column
CMP	Consent Management Platform: the cookie banner
TCF / `__tcfapi`	IAB Transparency & Consent Framework. The standard JavaScript API scripts use to read consent.
Stub	The tiny early version of `__tcfapi` that queues calls until the real CMP has loaded
GTM / tag manager / dataLayer	A tool that loads third-party scripts, and the array of events it reads
CSP	Content-Security-Policy: an HTTP header saying which origins may load scripts etc.
CLS	Cumulative Layout Shift: how much the page jumps while loading
Brand-safety zone	Green / yellow / red: whether commercial links may appear next to an article
S5, S6, ...	Rainmaker services from the Solution Architecture (S5 Content Ingestion, S6 Content & Intent)
MDC	Mapped Diagnostic Context: per-request values (like `trace_id`) added to every log line
ECS	Elastic Common Schema: the JSON log format used
Bean	An object Spring creates once and injects where needed
Fail fast / fail open	Stop at once on bad input (start-up) / keep the page working when an add-on fails (browser)